COMP338 - ARTIFICIAL INTELLIGENCE

Breaking a 32-bit Binary Passcode Using Genetic Algorithms

Student Name: Jamael Ghazawneh **Student ID:** 1210387

Student Name: Shahd Alhih **Student ID:** 1221817

Abstract

Genetic Algorithms are evolutionary optimization techniques inspired by the process of natural selection [1]. In this project, a Genetic Algorithm is implemented to guess a randomly generated 32-bit binary passcode. The algorithm starts with an initial population of random candidate solutions and iteratively evolves them using selection, crossover, and mutation operators [2]. The fitness of each candidate is evaluated based on the number of matching bits with the target passcode. Multiple experiments were conducted using different mutation rates to analyze their effect on the convergence speed of the algorithm [3]. The results demonstrate that an appropriate mutation rate significantly improves convergence efficiency and reduces the number of generations required to find the correct passcode.

Table Of Contents

Abstract I
Table Of Contents II
List of Figures III
List of Tables IV
1. Introduction 5
2. Background on Genetic Algorithms 5
2.1 Main Steps of a Genetic Algorithm 5
2.1.1 Initialization 5
2.1.2 Fitness Evaluation 6
2.1.3 Selection 6
2.1.4 Crossover 6
2.1.5 Mutation 6
2.1.6 Elitism 6
2.1.7 Termination 7
2.2 Genetic Algorithm Parameters 7
2.2.1 Population Size 7
2.2.2 Mutation Rate 7
2.2.3 Crossover Rate 7
2.2.4 Selection Method 7
2.2.5 Elitism Rate 7
3. Problem Formulation 8
4. Genetic Algorithm Implementation 8
5. Parameter Tuning Analysis 9
5.1 Effect of Mutation Rate 9
Figure 1: Convergence rate of the Genetic Algorithm showing the best fitness score per generation for different mutation rate values 10
5.2 Effect of Crossover Rate 10
Figure 2: Convergence rate of the Genetic Algorithm showing the effect of low and high crossover rates. 11
5.3 Effect of Population Size 11
Figure 3: Convergence rate of the Genetic Algorithm showing the effect of different population sizes. 11
6. Results and Discussion 11
6.1 Results for Mutation Rate 12
6.2 Results for Crossover Rate 12
6.3 Results for Population Size 12
7. Screenshots for the Run 13
7.1 Console Output for Crossover Rate and Population Size Experiments 13
Figure 4: Console output showing the convergence results for different crossover rates and population sizes. 13
7.2 Console Output for Mutation Rate Experiments 13
Figure 5: Console output showing the convergence results for different mutation rate values. 13
8. References 14

List of Figures

Figure 1: Convergence rate of the Genetic Algorithm showing the best fitness score per generation for different mutation rate values 10
Figure 2: Convergence rate of the Genetic Algorithm showing the effect of low and high crossover rates. 11
Figure 3: Convergence rate of the Genetic Algorithm showing the effect of different population sizes. 11
Figure 4: Console output showing the convergence results for different crossover rates and population sizes. 13
Figure 5: Console output showing the convergence results for different mutation rate values. 13

List of Tables

Table 1 : Description of the Genetic Algorithm workflow. 8
Table 2 : Results of the Genetic Algorithm for different mutation rate values. 12

1. Introduction

Genetic Algorithms are search and optimization techniques inspired by the process of natural evolution [1], [2]. They are widely used to solve complex problems where traditional deterministic methods may fail or become computationally expensive [4]. These algorithms operate by evolving a population of candidate solutions through selection, crossover, and mutation operators [1].

Password and passcode guessing problems represent a challenging search space, especially when the solution is generated randomly [5].
In this project, a Genetic Algorithm is applied to simulate the process of breaking a randomly generated 32-bit binary passcode. The algorithm begins with an initial population of random candidate solutions and iteratively improves them until the correct passcode is discovered.

The main objective of this project is to implement the Genetic Algorithm from scratch, analyze its convergence behavior, and investigate the impact of different parameter settings on the overall performance of the algorithm [3].

2. Background on Genetic Algorithms

Genetic Algorithms (GAs) are adaptive search and optimization techniques inspired by the process of natural evolution [1], [2]. They are widely used to solve complex optimization problems where traditional deterministic methods may not be effective [4]. A Genetic Algorithm works by evolving a population of candidate solutions over multiple generations using biologically inspired operators such as selection, crossover, and mutation [6].

In this project, a Genetic Algorithm is applied to simulate an attempt to break a 32-bit binary passcode. Each candidate solution represents a possible passcode, and the algorithm iteratively improves the population until the correct passcode is found.

2.1 Main Steps of a Genetic Algorithm

2.1.1 Initialization

The algorithm starts by generating an initial population of candidate solutions randomly to ensure sufficient diversity [6]. In this implementation, each individual is represented as a binary chromosome of length 32 bits, where each bit corresponds to a gene. Random initialization helps the algorithm explore the solution space effectively and reduces the risk of premature convergence.

2.1.2 Fitness Evaluation

Each individual is evaluated using a fitness function that measures the quality of the solution [1]. The fitness function is defined as the number of matching bits between the chromosome and the target 32-bit passcode. The fitness value ranges from 0 to 32, where higher values indicate better solutions.

2.1.3 Selection

Selection is the process of choosing individuals to act as parents for the next generation [7]. In this project, the population is sorted according to fitness values, and the top-performing half of the population is selected as a pool of potential parents. Parents are then chosen randomly from this pool, giving fitter individuals a higher probability of reproduction while maintaining genetic diversity.

2.1.4 Crossover

Crossover combines genetic material from two parent solutions to produce new offspring [1], [7]. A single-point crossover technique is used, where a random crossover point is selected, and parts of the chromosomes are exchanged between parents. The application of crossover is controlled by a predefined crossover rate.

2.1.5 Mutation

Mutation introduces small random changes to offspring chromosomes to maintain genetic diversity within the population [3], [8]. In this implementation, mutation is performed using a bit-flip mechanism, where individual bits are inverted with a certain probability. Mutation helps the algorithm avoid stagnation and improves exploration of the search space.

2.1.6 Elitism

Elitism ensures that the best-performing individuals are preserved across generations [6]. The top two individuals with the highest fitness values are directly copied to the next generation, preventing the loss of high-quality solutions during reproduction.

2.1.7 Termination

The algorithm terminates when a predefined stopping condition is satisfied [6]. In this project, termination occurs when the fitness value of the best individual reaches 32, indicating that the correct 32-bit passcode has been successfully discovered.

2.2 Genetic Algorithm Parameters

2.2.1 Population Size

Population size refers to the number of individuals in each generation [6]. Larger populations generally increase genetic diversity and improve solution quality but require more computational resources.

2.2.2 Mutation Rate

The mutation rate defines the probability of altering a gene during mutation [3], [8]. Low mutation rates may slow exploration, while high mutation rates may disrupt good genetic structures.

2.2.3 Crossover Rate

The crossover rate determines the probability of applying crossover between selected parents [7]. Appropriate crossover rates help efficiently combine useful genetic material and speed up convergence.

2.2.4 Selection Method

The selection method specifies how parent individuals are chosen for reproduction [7]. Fitness-based selection methods favor individuals with higher fitness values while still allowing diversity within the population.

2.2.5 Elitism Rate

The elitism rate indicates the number of top-performing individuals preserved in each generation [6]. Preserving elite individuals ensures that solution quality is maintained throughout the evolutionary process.

3. Problem Formulation

In this project, each chromosome is represented as a binary string of length 32, where each gene corresponds to a single bit with a value of either 0 or 1 [1]. This representation allows the Genetic Algorithm to model candidate solutions in a simple and efficient manner.

The target passcode is a randomly generated 32-bit binary sequence. The fitness of each chromosome is evaluated by comparing it to the target passcode and counting the number of matching bits at corresponding positions [4]. A higher fitness value indicates a closer approximation to the correct passcode, with the maximum fitness value achieved when all 32 bits match.

4. Genetic Algorithm Implementation

The Genetic Algorithm was implemented from scratch following the standard evolutionary computation process [6], [7]. Table 1 summarizes the main steps of the algorithm and their roles in solving the passcode guessing problem.

Table 1: Description of the Genetic Algorithm workflow.

Step	Description
Initialization	Generate an initial population of random 32-bit binary chromosomes [1].
Fitness Evaluation	Evaluate each chromosome by counting the number of matching bits with the target passcode [4].
Selection	Select parent chromosomes from the top half of the population based on fitness values [7].
Crossover	Apply single-point crossover to generate offspring from selected parents [7].
Mutation	Randomly flip a single bit in the offspring with a predefined mutation rate [3], [8].
Elitism	Preserve the best-performing chromosomes and carry them to the next generation [6].
Termination	Stop the algorithm when a chromosome reaches the maximum fitness value (32) [6].

This structured implementation ensures a balance between exploration and exploitation of the search space [8], allowing the algorithm to efficiently converge toward the correct passcode.

5. Parameter Tuning Analysis

To study the effect of tuning the Genetic Algorithm (GA) parameters on the convergence behavior, several experiments were conducted by varying **one parameter at a time** while keeping all other settings constant. This approach ensures that any observed change in performance is mainly caused by the modified parameter only [3], [8].
In all experiments, the chromosome length was fixed to 32 bits, the same randomly generated target passcode was used, and the fitness function, selection method, crossover method, and termination condition remained unchanged.

5.1 Effect of Mutation Rate

In this experiment, the **mutation rate** was varied while keeping all other GA parameters fixed. Four mutation rate values were tested: **0.005, 0.01, 0.02, and 0.05**. For each run, the best fitness score was recorded at every generation until the optimal solution (fitness = 32) was reached.

Figure 1 shows the convergence rate of the Genetic Algorithm for different mutation rates. The results indicate that the mutation rate has a noticeable effect on the speed of convergence. Very low mutation rates may slow convergence due to limited exploration and reduced introduction of new genetic variations [8]. Conversely, higher mutation rates increase randomness, which may disrupt good solutions and reduce convergence stability [6], [8]. A moderate mutation rate provides a balance between exploration and exploitation, leading to faster and more stable convergence [3].

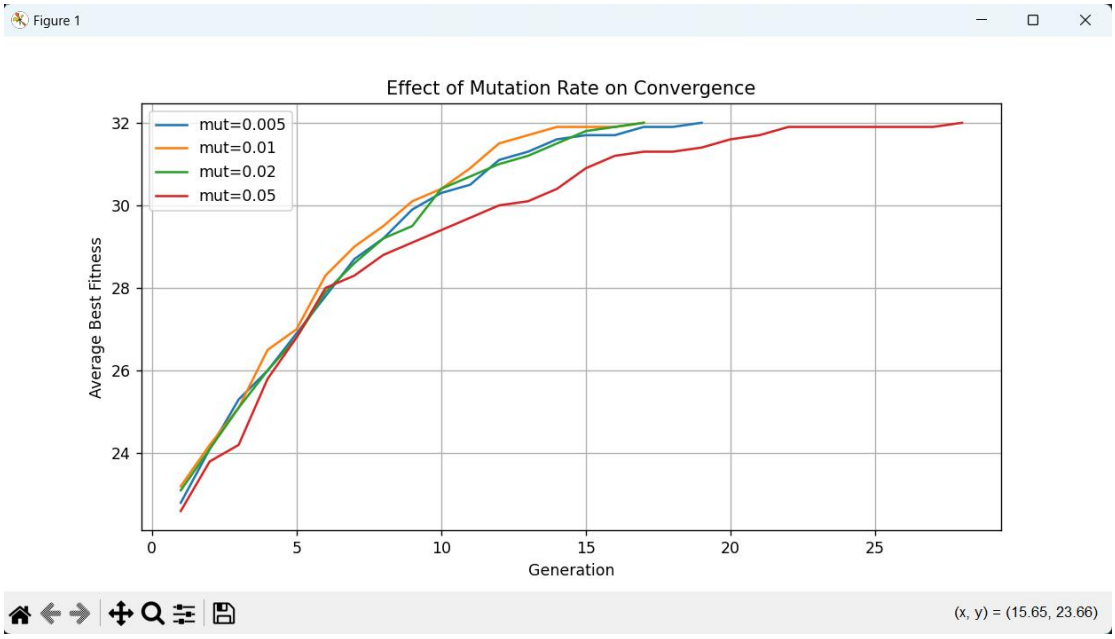

Figure 1: Convergence rate of the Genetic Algorithm showing the best fitness score per generation for different mutation rate values.

5.2 Effect of Crossover Rate

In this experiment, the **crossover rate** was varied while keeping the mutation rate, population size, and all other parameters constant. Two crossover rate values were tested: **0.3 (low crossover)** and **0.9 (high crossover)**. The same target passcode was used, and the best fitness score was recorded at each generation.

Figure 2 illustrates the effect of the crossover rate on the convergence behavior of the Genetic Algorithm. The results show that a higher crossover rate leads to faster convergence compared to a lower crossover rate. This is because a higher crossover rate allows more effective recombination of genetic material between parent chromosomes, increasing diversity and accelerating the discovery of better solutions [3].

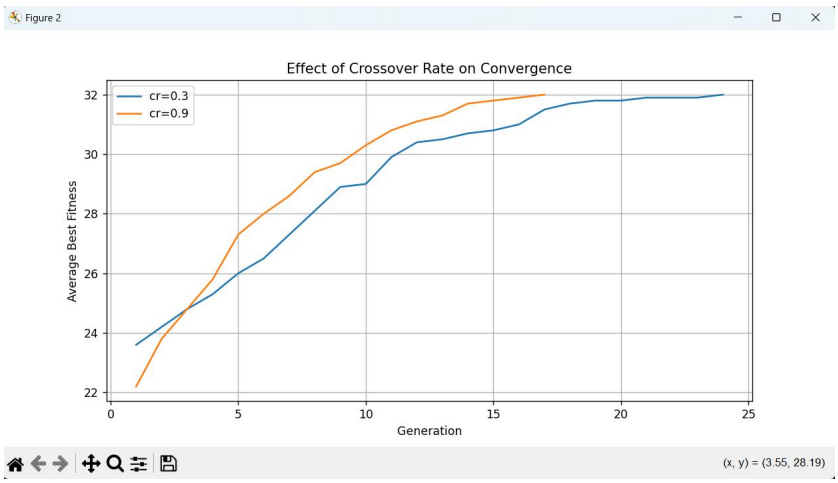

Figure 2: Convergence rate of the Genetic Algorithm showing the effect of low and high crossover rates.

5.3 Effect of Population Size

In this experiment, the **population size** was varied while keeping the mutation rate and crossover rate fixed. Two population sizes were tested: **50 (small population)** and **200 (large population)**. The convergence behavior was measured by recording the best fitness value per generation.

Figure 3 shows the convergence rate for different population sizes. The results indicate that a larger population size converges faster than a smaller population size. This is mainly due to increased genetic diversity in larger populations, which improves exploration of the search space and reduces the risk of premature convergence. However, larger populations may increase computational cost, even though they often achieve better convergence performance [8].

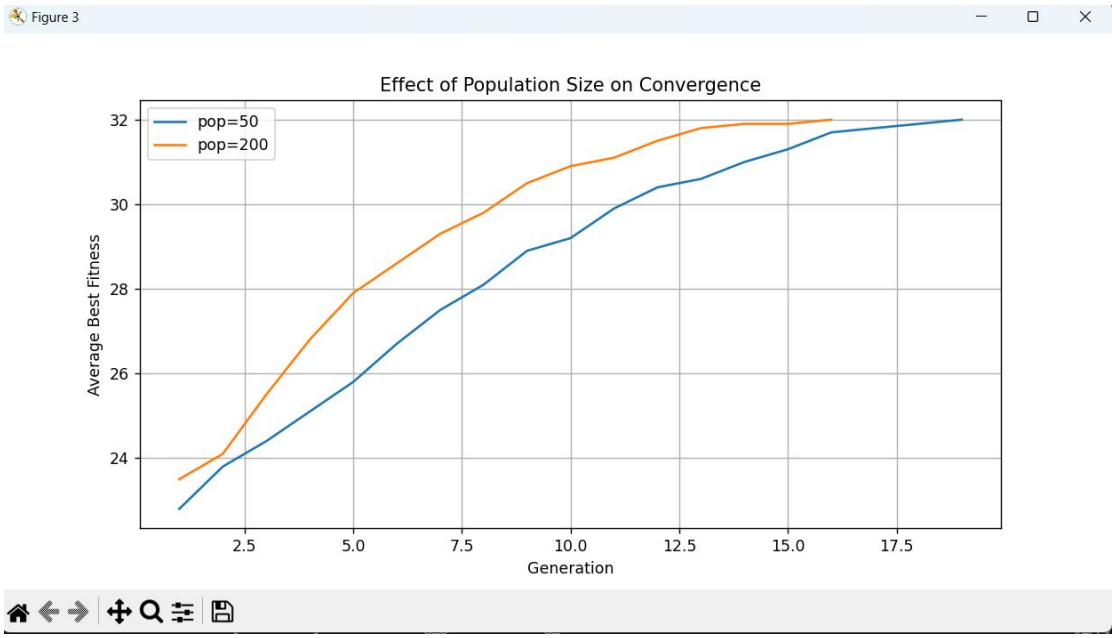

Figure 3: Convergence rate of the Genetic Algorithm showing the effect of different population sizes.

6. Results and Discussion

The Genetic Algorithm successfully converged to the correct 32-bit binary passcode in all conducted experiments. The results demonstrate that the algorithm is capable of efficiently solving the passcode guessing problem using evolutionary search techniques [4], [6]. Convergence was achieved when the best fitness score reached the maximum value of 32, indicating that the correct passcode had been found.

when the best fitness score reaches the maximum value of 32, indicating that the correct passcode has been found.

6.1 Results for Mutation Rate

Table 2 presents the number of generations and execution time required for the algorithm to reach convergence for different mutation rate values.

Table 2: Results of the Genetic Algorithm for different mutation rate values.

Mutation Rate	Generations to Converge	Time (seconds)
0.005	14	0.0040
0.01	15	0.0040
0.02	13	0.0000
0.05	13	0.0030

From the results shown in Table 2, it can be observed that different mutation rates lead to different convergence behaviors. Lower mutation rates tend to require more generations to reach the optimal solution because they limit the introduction of new genetic variations, thereby reducing exploration of the search space [8]. Conversely, higher mutation rates introduce more randomness, which may accelerate convergence but can also affect solution stability [6].
In this experiment, mutation rates of 0.02 and 0.05 achieved convergence in fewer generations compared to lower mutation rates, confirming the importance of selecting an appropriate mutation rate to balance exploration and exploitation [3], [8].

6.2 Results for Crossover Rate

The effect of the crossover rate on convergence was evaluated by testing two values: a low crossover rate (0.3) and a high crossover rate (0.9), while keeping all other parameters fixed. The experimental results, illustrated in Figure 2, show that a higher crossover rate leads to faster convergence.

A higher crossover rate improves the recombination of genetic material between parent chromosomes, allowing useful gene combinations to propagate more effectively. As a result, the algorithm reaches the optimal solution in fewer generations when a higher crossover rate is used [3].

6.3 Results for Population Size

The impact of population size on convergence was analyzed by comparing a small population (50 individuals) with a larger population (200 individuals). As shown in Figure 3, the algorithm converges faster when a larger population size is used.

Larger populations provide higher genetic diversity, which enhances exploration of the search space and reduces the likelihood of premature convergence. However, increasing the population size may also increase computational cost, even though it generally leads to improved convergence performance [8].

7. Screenshots for the Run

This section presents screenshots captured during the execution of the Genetic Algorithm program. The screenshots show the runtime output of the algorithm and summarize the experimental results obtained for different parameter settings.

7.1 Console Output for Crossover Rate and Population Size Experiments

This screenshot shows the console output produced during the execution of the Genetic Algorithm when testing different **crossover rate** and **population size** values. It reports the number of generations required to reach convergence and the execution time for each configuration.

```
(.venv) PS C:\Users\cts\Desktop\Genatic Algorithm> & "C:/Users/cts/Desktop/Genatic Algorithm/.venv/S
ripts/python.exe" "c:/Users/cts/Desktop/Genatic Algorithm/genatic.py"
============================================================
[Crossover] rate=0.3  avg_gens=17.20 avg_time=0.0055s
[Crossover] rate=0.9  avg_gens=13.70 avg_time=0.0048s
============================================================
[Population] size=50  avg_gens=15.70 avg_time=0.0027s
[Population] size=200 avg_gens=12.00 avg_time=0.0085s
```

Figure 4: Console output showing the convergence results for different crossover rates and population sizes.

7.2 Console Output for Mutation Rate Experiments

This screenshot presents the console output obtained from running the Genetic Algorithm with different **mutation rate** values. The output includes the target passcode, the number of generations required to reach convergence, and the execution time for each mutation rate.

```
(.venv) PS C:\Users\cts\Desktop\Genatic Algorithm> & "C:/Users/cts/Desktop/Genatic Algorithm/.venv/Sc
ripts/python.exe" "c:/Users/cts/Desktop/Genatic Algorithm/genatic.py"
Target: 00101111001011011001000010100110
============================================================
[Mutation] rate=0.005  avg_gens=14.10 avg_time=0.0050s
[Mutation] rate=0.01   avg_gens=12.70 avg_time=0.0045s
[Mutation] rate=0.02   avg_gens=13.90 avg_time=0.0059s
[Mutation] rate=0.05   avg_gens=19.50 avg_time=0.0091s
============================================================
```

Figure 5: Console output showing the convergence results for different mutation rate values.

8. References

[1] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*, Reading, MA: Addison-Wesley, 1989.

[2] J. H. Holland, *Adaptation in Natural and Artificial Systems*, Ann Arbor, MI: University of Michigan Press, 1975.

[3] A. E. Eiben and J. E. Smith, *Introduction to Evolutionary Computing*, Berlin, Germany: Springer, 2003.

[4] M. Mitchell, *An Introduction to Genetic Algorithms*, Cambridge, MA: MIT Press, 1996.

[5] S. Naik and R. Chavan, "Optimization of software testing using genetic algorithm," *International Journal of Soft Computing and Engineering*, vol. 3, no. 2, pp. 88-91, May 2013.

[6] Z. Michalewicz, *Genetic Algorithms + Data Structures = Evolution Programs*, 3rd ed., Berlin, Germany: Springer-Verlag, 1996.

[7] T. Bäck, D. B. Fogel, and Z. Michalewicz, Eds., *Handbook of Evolutionary Computation*, Oxford, UK: Oxford University Press, 1997.

[8] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*, Chichester, UK: John Wiley & Sons, 2001.